Lecture 12: Goal-Conditioned RL. Unsupervised RL

Petr Kuderov

Researcher at MIPT & AIRI

April 22, 2026

Why Goal-Conditioned RL?

Standard RL:

- one task
- one reward function
- one policy

Goal-conditioned RL:

- family of tasks
- varying target state / observation / configuration
- one policy reused across goals

Condition policy and value on goal:

$$\pi(a \mid s, g), \quad Q(s, a, g), \quad V(s, g)$$

Examples:

- navigation to target position
- robot reaching target pose
- pushing object to target location
- agent follows a textual instruction in a video game

GCRL Setup

MDP with goal variable $g \in \mathcal{G}$.

Trajectory objective for a fixed goal:

$$J(\pi; g) = \mathbb{E}_{\tau \sim \pi(.|g)} \left[\sum_t \gamma^t r(s_t, a_t, s_{t+1}, g) \right]$$

Most common sparse reward:

$$r_t = 1[f(s_{t+1}) \approx g] - 1$$

or distance-based shaping:

$$r_t = -d(f(s_{t+1}), g)$$

Key ingredients:

- goal space $\mathcal{G}$
- reward definition wrt goal
- goal distribution $p(g)$
- policy/value conditioned on g

Why Is It Difficult?

Sparse reward + large goal space $\Rightarrow$ cold start is hard

For many trajectories:

- desired goal never reached
- reward almost always identical
- little signal for improvement

In deep RL, exploration must solve two problems:

- discover useful states
- discover them for many goals

Naive conditioning is easy to write, hard to optimize.

UVFA: Direct Extension of Value Learning

Universal Value Function Approximator (UVFA):

$$Q(s, a, g) \approx Q^*(s, a, g)$$

Bellman target:

$$y = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a', g)$$

Deterministic actor-critic version:

$$y = r_t + \gamma Q_{\theta^-}\Big(s_{t+1}, \mu_{\varphi^-}(s_{t+1}, g), g\Big)$$

$$\nabla_\varphi J \approx \mathbb{E}\Big[\nabla_a Q_{\theta(s,a,g)} \mid_{a=\mu_{\varphi(s,g)}} \nabla_\varphi \mu_{\varphi(s,g)}\Big]$$

Interpretation:

- same state, different goals $\rightarrow$ different optimal actions
- generalization across goals through shared network

UVFA in Point Maze

Inputs:

- state $s_t = (x_t, y_t, ...)$
- goal $g = (x^*, y^*)$
- actor / critic receive concatenated (s, g)

Example sparse reward:

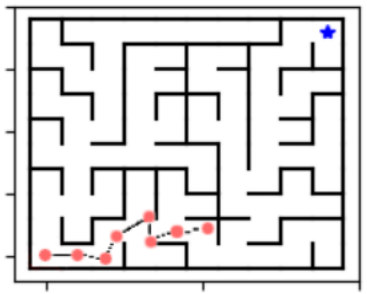
$$r_t = 1[\| p_{t+1} - g \| \leq \varepsilon] - 1$$

TD3-style critic target:

$$y = r_t + \gamma \min_{i \in \{1,2\}} Q_i^-(s_{t+1}, a', g)$$

with

$$a' = \mu^-(s_{t+1}, g) + \eta$$



What works well:

- simple implementation
- parameter sharing across goals
- one replay buffer for many tasks

What breaks:

- almost no positive rewards early on
- critic sees weak supervision
- actor follows noisy critic gradients

Failure Mode: Cold Start For Training

Early, we get [almost] only negative samples — unsuccessful trajectories.
Replay buffer under sparse rewards:

$$(s_t, a_t, s_{t+1}, g, r_t \approx -1)$$

for almost all transitions.

Then Bellman target becomes nearly constant:

$$y \approx -1 + \gamma Q(s_{t+1}, \mu(s_{t+1}, g), g)$$

Consequences:

- weak discrimination between actions
- poor credit assignment
- success states are rare and hard to propagate

> How can a failed episode still become useful for learning?

Hindsight Experience Replay (HER)

> Trajectory may fail for intended goal g, but it succeeded for some achieved goal g'

Relabel transition or episode with achieved goal:

$$(s_t, a_t, s_{t+1}, \boldsymbol{g}, \boldsymbol{r_t}) \rightarrow (s_t, a_t, s_{t+1}, \boldsymbol{g'}, \boldsymbol{r'_t})$$

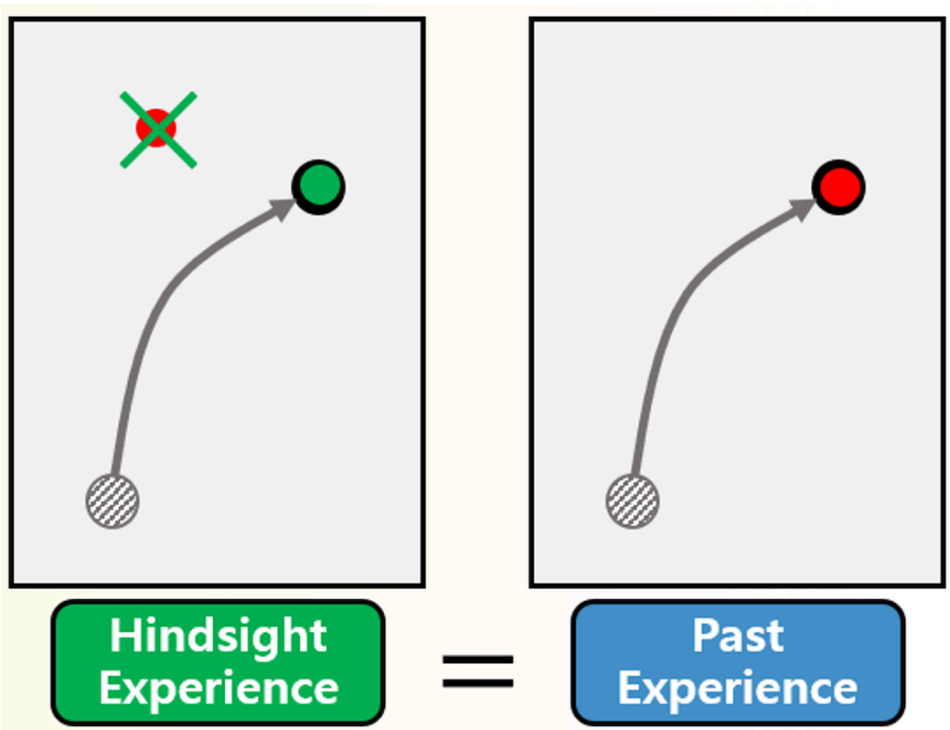
Usually, $g' = f(s_T)$ or a future achieved state.

Reward recomputed under relabeled goal:

$$r'_t = r(s_t, a_t, s_{t+1}, g')$$

Effect:

- failed rollouts become supervised signal
- dense source of successful examples
- implicit curriculum from own behavior



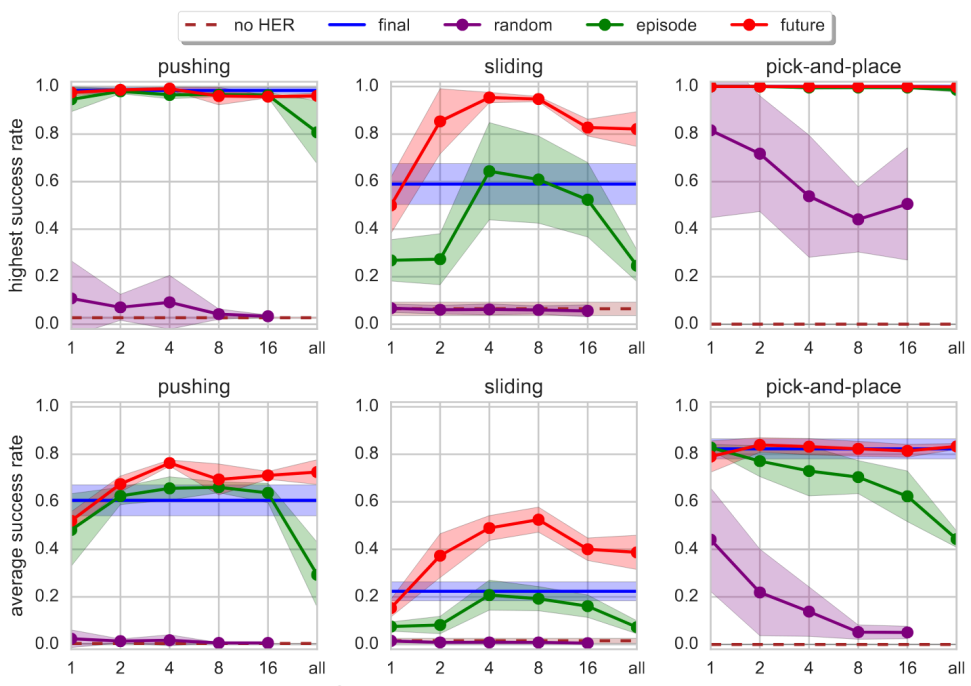
HER: Typical Relabeling Choices

Given episode $(s_0, ..., s_T)$:

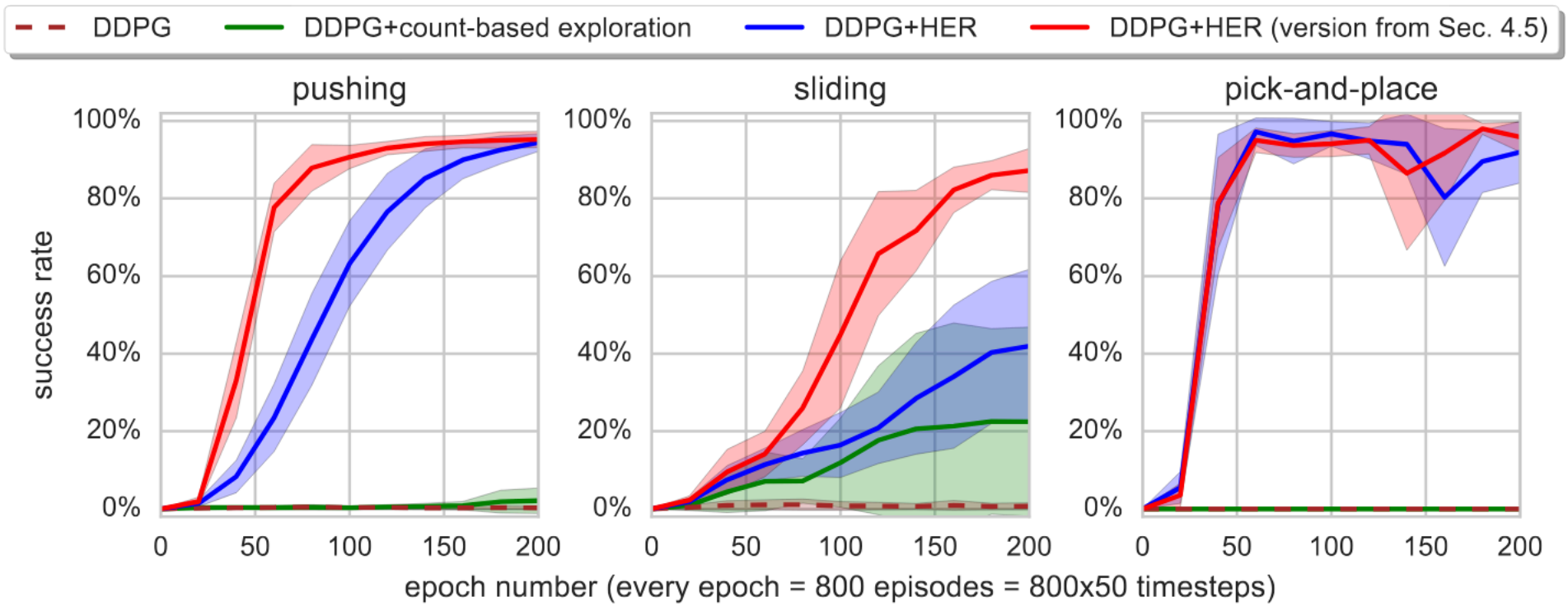
- final: $g' = f(s_T)$
- future: $g' = f(s_k)$ for $k > t$
- episode: g' from any achieved state

Training objective unchanged:

- same TD loss
- same actor loss
- only data distribution changes



HER: Results


DDPG
DDPG+count-based exploration
DDPG+HER
DDPG+HER (version from Sec. 4.5)
pushing
sliding
pick-and-place
success rate
100%
80%
60%
40%
20%
0%
0
50
100
150
200
epoch number (every epoch = 800 episodes = 800x50 timesteps)

What About On-Policy Algorithms?

HER is naturally off-policy:

- relabel past transitions
- reuse them many times
- learn under goals different from rollout goal

$\Rightarrow$ Can be easily plugged to DDPG, TD3, SAC, etc.

With on-policy methods:

- PPO / A2C cannot freely replay relabeled data [w/o importance sampling]
- HER-style reuse becomes much less natural
- We usually rely on other approaches:
  - reward shaping
  - goal curriculum / goal sampling
  - better goal representations
  - intrinsic or unsupervised objectives

> HER is a generic add-on, but only for off-policy learning.

Goal Sampling as a Separate Design Choice

So far goals were sampled from some $p(g)$.

Training objective averages over goals:

$$\max_{\pi} \mathbb{E}_{g \sim p(g)}[J(\pi; g)]$$

But choice of $p(g)$ strongly affects learning.

Uniform goal sampling:

- simple baseline
- often wastes samples on impossible or trivial goals

Idea:

- avoid only easy goals
- avoid only impossible goals
- adapt to current competence

Two views:

- coverage of state-goal space
- curriculum by difficulty / learning progress

Goal Sampling

Skew-Fit:

- learn goal distribution from visited states
- upweight rare states
- target broad state coverage

Informal objective:

$$p(g) \propto \frac{1}{p_{\text{data}}}(g)$$

> "visit everything" $\longrightarrow$ coverage
> "learn at your limit" $\longrightarrow$ difficulty
> "learn where you improve" $\longrightarrow$ progress

GoalGAN:

- generate goals, discriminate by difficulty
- $\Rightarrow$ generate goals of intermediate difficulty
- avoid too easy / too hard goals
- curriculum over feasible frontier

ALP-GMM:

- sample goals w/ high learning progress
- track where performance changes most

Learning progress signal:

$$\text{ALP}(g) \approx \mid c_{\text{new}}(g) - c_{\text{old}}(g) \mid$$

High ALP $\longrightarrow$ informative goals

Goal Representation: What Is a Goal?

So far:

- goal = raw target state
- compare directly in observation space

But this may be poor when:

- observations are high-dimensional
- Euclidean distance is semantically wrong
- relevant factors are hidden

Use representation $z = \varphi(s)$ and define goals in latent space: $z_t = \varphi(s_t), \quad z_g = \varphi(g)$

Reward in latent space:

$$r_t = - \parallel z_{t+1} - z_g \parallel$$

Goal-conditioned value now becomes:

$$Q(s, a, g) \quad \vee \quad Q(\varphi(s), a, \varphi(g))$$

Goal Representation Methods

RIG (Reinforcement Learning with Imagined Goals):

- learn latent space with VAE
- sample imagined goals in latent space
- compare current state and goal through encoder

Pipeline:

$$s \to \varphi(s) = z, \quad g \to \varphi(g) = z_g$$

Policy / critic conditioned on (z, z_g).

Distance-based view:

- learn representation where distance reflects reachability
- value behaves like negative distance-to-goal or steps-to-goal

Informally:

$$Q(s, a, g) \approx -d_{\text{dyn}}(s_{t+1}, g)$$

Benefits:

- better geometry of goals
- smoother generalization
- more meaningful similarity between states

Putting It Together: A Design Space

A goal-conditioned agent is defined by at least four choices.

1. Goal-conditioned backbone:
   - UVFA + TD3 in our notebook
2. Replay / relabeling:
   - none
   - HER with future or final strategy
3. Goal distribution:
   - uniform
   - curriculum / coverage based
4. Goal representation:
   - raw state goal
   - learned latent goal

Same policy class can behave very differently depending on these choices.

If Goals Are Not Given

> What if environment gives no external goals?

Then we can learn diverse behaviors first.

Skill-conditioned policy:

$$\pi(a \mid s, z)$$

where z is not a target state but a latent skill index / vector.

Now objective is not task reward.
Instead: discover distinguishable behaviors.

This is the bridge to unsupervised RL.

DIAYN: Diversity as an Objective

DIAYN (Diversity Is All You Need): *Learn skills z that lead to distinguishable states.*

Typical objective:

$$\max \mathbb{E}\left[r_{\text{ext}} + \alpha \log q_\psi(z \mid s) - \alpha \log p(z)\right]$$

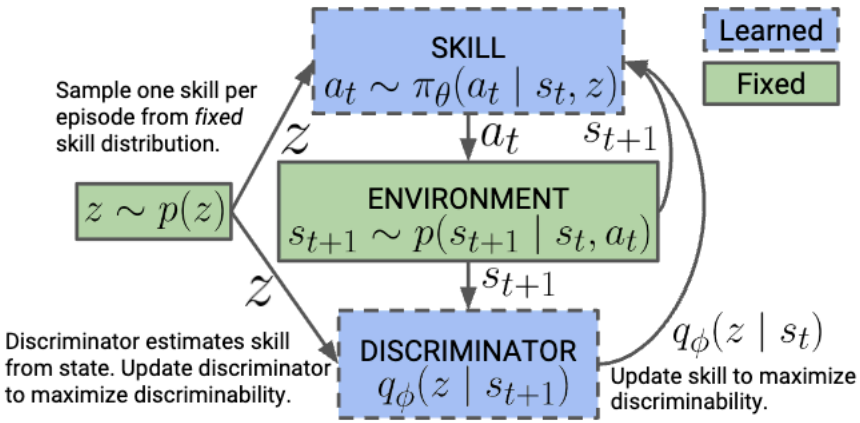
In pure unsupervised version:

- no external reward
- discriminator predicts skill from visited state
- policy encouraged to make skills diverse

Interpretation:

- skills partition state space
- each skill becomes reusable behavior primitive

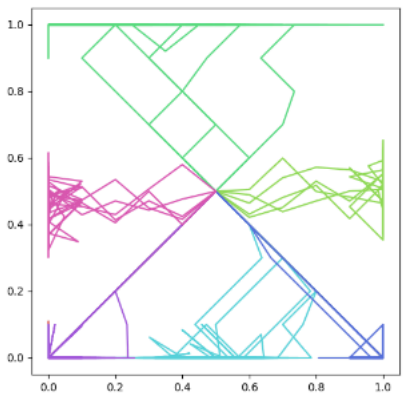
DIAYN skills



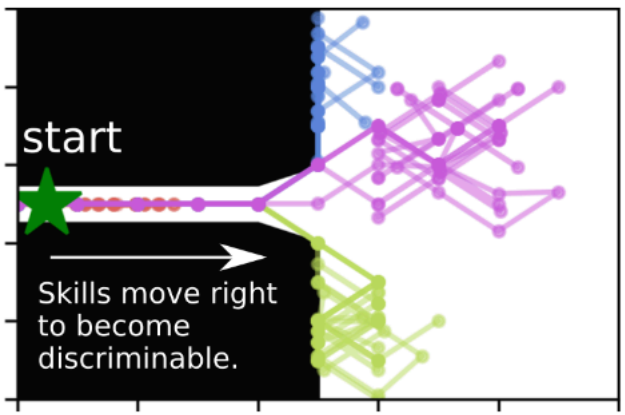
Algorithm 1: DIAYN

while *not converged* **do**
  Sample skill $z \sim p(z)$ and initial state $s_0 \sim p_0(s)$
  for $t \leftarrow 1$ **to** *steps_per_episode* **do**
    Sample action $a_t \sim \pi_\theta(a_t \mid s_t, z)$ from skill.
    Step environment: $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$.
    Compute $q_\phi(z \mid s_{t+1})$ with discriminator.
    Set skill reward $r_t = \log q_\phi(z \mid s_{t+1}) - \log p(z)$
    Update policy (θ) to maximize r_t with SAC.
    Update discriminator (ϕ) with SGD.

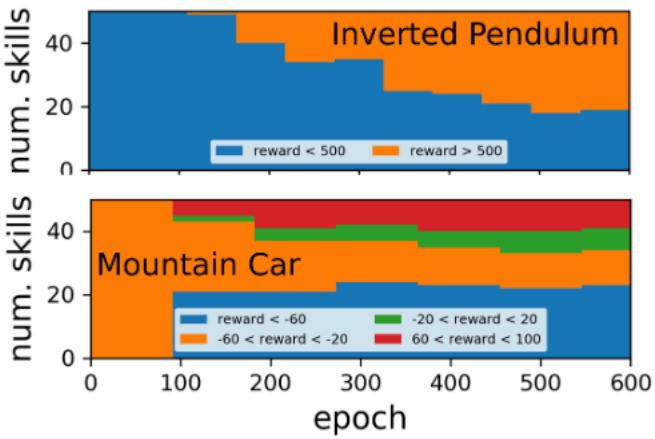
DIAYN skills



(a) 2D Navigation



(b) Overlapping Skills



(c) Training Dynamics

METRA and Geometry of Skills

Idea:

- learn skills with a structured latent geometry
- representation should reflect controllable transitions

High-level view:

- latent variable indexes behavior
- behavior induces predictable displacement in representation space

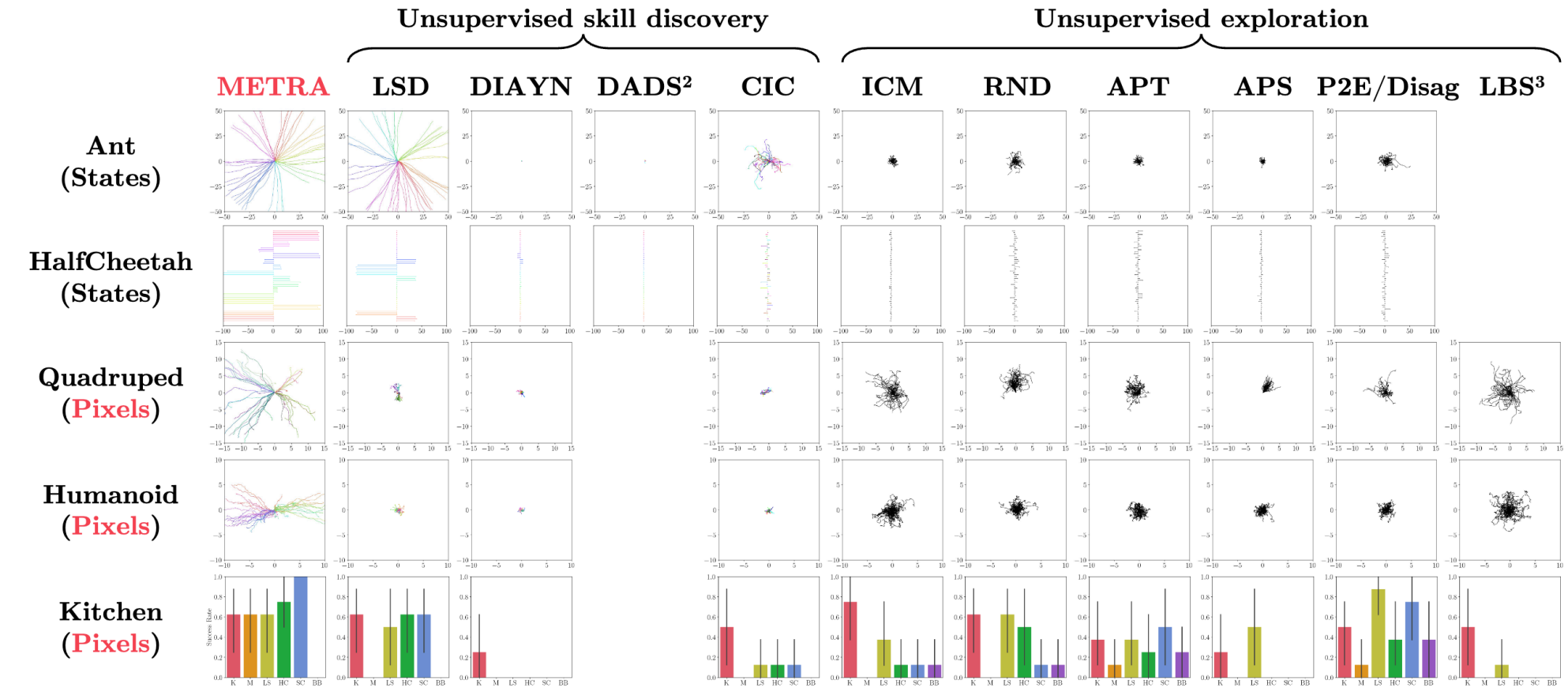
This connects to goal-conditioned RL:

- goals specify desired state / representation
- skills specify consistent directions of behavior

Thus goals and skills are somewhat dual:

- goal-conditioned RL: given target, learn behavior
- unsupervised RL: learn behaviors, then interpret / reuse them as targets or primitives

DIAYN, METRA and Others



Outro

Goal-conditioned RL:

- one policy, many goals
- solves multi-goal sparse-reward tasks

Main ideas:

- UVFA gives formulation
- HER fixes supervision problem
- goal sampling fixes data distribution
- representation learning fixes geometry of goals
- unsupervised RL hints what to do when goals are not given